

Concurrent Promises

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Sequential Costs the Sum

- Awaiting one, then the next, runs them **one at a time**

```
const a = await fetchUser(1); // 1s
const b = await fetchUser(2); // + 1s
const c = await fetchUser(3); // + 1s
// ~3 seconds total
```

- Correct when each step **needs** the previous result
- Wasteful when the three jobs are independent

Concurrent Costs the Slowest

- Start them all, then wait for the group

```
const [a, b, c] = await Promise.all([
  fetchUser(1),
  fetchUser(2),
  fetchUser(3),
]);
// ~1 second total
```

- The promises start the moment they are **created**, not when awaited
- `Promise.all` just waits for the whole array

Results Come Back in Order

```
const [slow, fast] = await Promise.all([
  delay(300, "slow"),
  delay(50, "fast"),
]);

console.log(slow, fast); // "slow" "fast"
```

- The array order matches the order you **passed in**
- Not the order they finished, so destructuring is always safe

Promise.all Is All-or-Nothing

```
try {
  const results = await Promise.all([
    ok("a"),
    Promise.reject(new Error("boom")),
    ok("c"),
  ]);
} catch (error) {
  console.log(error.message); // "boom"
}
```

- **One rejection rejects the whole thing**, immediately
- You lose the results that *did* succeed
- The other promises are not canceled; they just finish unnoticed

Promise.allSettled Never Short-Circuits

```
const results = await Promise.allSettled([
  ok("a"),
  Promise.reject(new Error("boom")),
]);

console.log(results);
// [ { status: "fulfilled", value: "a" },
//   { status: "rejected", reason: Error: boom } ]
```

- Always fulfills, with one outcome object per promise
- Use it when partial success is still useful, a dashboard of widgets

Reading `allSettled` Results

```
const ok = results.filter((r) => r.status === "fulfilled");
const bad = results.filter((r) => r.status === "rejected");

console.log(`${ok.length} of ${results.length} loaded`);

const values = ok.map((r) => r.value);
const reasons = bad.map((r) => r.reason.message);
```

- Fulfilled entries carry `value`; rejected ones carry `reason`

Promise.race and Promise.any

```
// race - first to SETTLE wins, success or failure
const winner = await Promise.race([slowFetch(), timeout(5000)]);

// any - first to SUCCEED wins; ignores rejections
const fastest = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

- `race` is the timeout pattern: whichever finishes first
- `any` rejects only if **every** promise rejects (an `AggregateError`)

Choosing the Right One

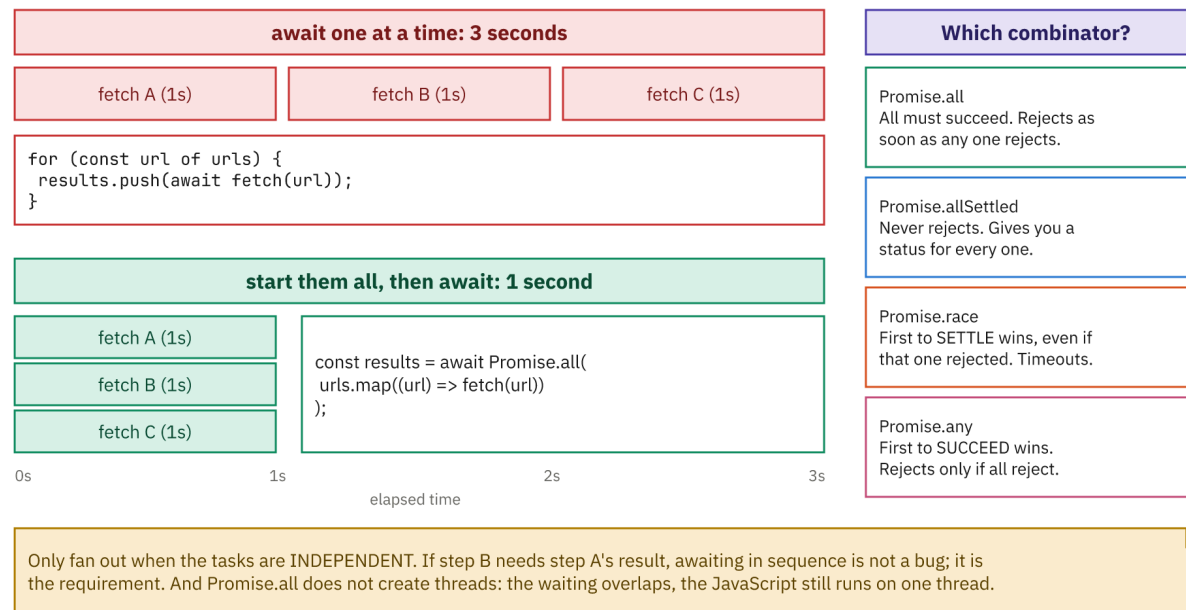
You need	Use
All of them, fail fast	<code>Promise.all</code>
Every outcome, good and bad	<code>Promise.allSettled</code>
First to finish, either way	<code>Promise.race</code>
First success, ignore failures	<code>Promise.any</code>
Each step needs the last	plain sequential <code>await</code>

- Reach for `all` by default; `allSettled` when a failure is survivable

Choosing a Combinator

Sequential vs Concurrent

Awaiting in a loop is a queue. `Promise.all` is a starting gun for tasks that do not depend on each other.



- `all` needs every one; `allSettled` never rejects
- `race` takes the first to settle; `any` the first to succeed